Telematics Platform User Guide

TP common library

Document Information	
Filename	TP_F101U-00023_User_Guide_Common_Library
Status	Release
Revision Date	Oct 01 , 2020
Version	03.00
Document ID	TP_F101U-00023

Standard Proprietary Terms

THE REPRODUCTION, DISTRIBUTION AND UTILIZATION OF THIS DOCUMENT AS WELL AS THE COMMUNICATION OF ITS CONTENTS TO OTHERS WITHOUT EXPRESS AUTHORIZATION IS PROHIBITED. OFFENDERS WILL BE HELD LIABLE FOR THE PAYMENT OF DAMAGES. ALL RIGHTS RESERVED IN THE EVENT OF THE GRANT OF A PATENT, UTILITY MODEL OR DESIGN.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

Table of contents

1 Introduction 4
1.1 Notational Conventions 4
2 Setup 5
2.1 Tools/Equipment Needed 5
2.2 Setup Instructions 5
2.3 Setup Troubleshooting 5
3 Features/Components 6
3.1 Platform IPC framework 6
3.1.1 Overview 6
3.1.1.1 Platform Content Overview 6
3.1.1.2 Product Expectations 6
3.1.1.3 Extending Platform Functionality 7
3.1.2 Setup 7
3.1.2.1 Tools/Equipment Needed 7
3.1.2.2 Setup Instructions 7
3.1.2.3 Setup Troubleshooting 7
3.1.3 Usage 7
3.1.3.1 Basic Instructions 7
3.1.3.2 Detailed Instructions 7
3.1.3.3 Testing/Error Reporting 8
3.2 Platform service monitor framework 9
3.2.1 Overview 9
3.2.1.1 Platform Content Overview 9
3.2.1.2 Product Expectations 10
3.2.1.3 Extending Platform Functionality 10
3.2.2 Setup 10
3.2.2.1 Tools/Equipment Needed 11
3.2.2.2 Setup Instructions 11
3.2.2.3 Setup Troubleshooting 11
3.2.3 Usage 11
3.2.3.1 Basic Instructions 11
3.2.3.2 Detailed Instructions 11
3.2.3.3 Testing/Error Reporting 16
3.3 Miscellaneous Interfaces for debugging 16
3.3.1 Overview 16
3.3.1.1 Platform Content Overview 16
3.3.1.2 Product Expectations 16
3.3.1.3 Extending Platform Functionality 16
3.3.2 Setup 17

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

3.3.2.1 Tools/Equipment Needed 17
3.3.2.2 Setup Instructions 17
3.3.2.3 Setup Troubleshooting 17
3.3.3 Usage 17
3.3.3.1 Basic Instructions 17
3.3.3.2 Detailed Instructions 17
4 Known Issues 19
5 FAQs 19
6 Troubleshooting/Tips and Tricks 19
7 References 19
7.1 Abbreviations/Acronyms/Definitions 19
7.2 Platform Documentation 19
7.3 Website links 19
8 Revision History 20

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

1 Introduction

This document provides usage information for Telematics Platform (TP) middleware common library APIs.

TP middleware common APIs are packaged in user space shared object, and exposed to be used by other middleware softwares and any 3rd party application software.

Applications may use these APIs because these platform APIs abstract underlying implementation and facilitates applications to use various platform functionalities in a uniform manner, among different modules. The design has built in support for performance and robustness. Moreover, using common library helps in debugging and system log generation in a consistent way.

This shared library currenty provides APIs for system wide Inter Process Communication (IPC) setup related functionality, system service availability notification functionality, APIs for debugging aid, setting up watchdog etc.

1.1 Notational Conventions

Ctrl+x	*A sequence with a '+' indicates that you must press and hold one key or pointing device button while pressing another key or pointing device button*
[ ]	*Optional elements in a command*
…..	*Indicates one of the following possibilities:* • *Additional optional arguments in a statement have been omitted* • *The preceding item or items can be repeated one or more times* • *Additional parameters, values, or other information can be entered*
-	*Line continuation: indicates that a command line or code line continues on the following line when at the end of a line*
Italic text	*Indicates important information, complete titles of manuals or variables*
`Mono-spaced`	*Example code or interactive screen display/output*
Numbers	*All numbers are assumed to be decimal unless otherwise noted. Non decimal radixes – binary, octal or hexadecimal – are explicitly indicated*
Underline blue text	A link to an external reference (e.g., website, document)

`Modules`
`|-> section`
`|-> item`
	A link to a section or item in the Telematics Platform html interface specification (Doxygen output from platform code headers)

Diagram Legend

- Telematics Platform
- Product Specific
- 3rd Party

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

2 Setup

2.1 Tools/Equipment Needed

Not Applicable.

2.2 Setup Instructions

HOWTO Use TP Common Library

1) Use pkg-config in the application Makefile to compile and link with the TP common library

```
LDFLAGS += `pkg-config --libs tpsys`
CFLAGS += `pkg-config --cflags tpsys`
```

2) Include public tpsys header in application source code

```
#include "tpsys.h"
```

HOW TO Use TP Service Monitor Library

1) Use pkg-config in the application Makefile to compile and link with the TP common library

```
INCLUDE += `pkgconfig --cflags libcas-svcmon`
LDFLAGS += `pkg-config --libs libcas-svcmon`
```

2) Include public tpsys header in application source code

```
#include "tpSVCMON.h"
```

2.3 Setup Troubleshooting

Not Applicable.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

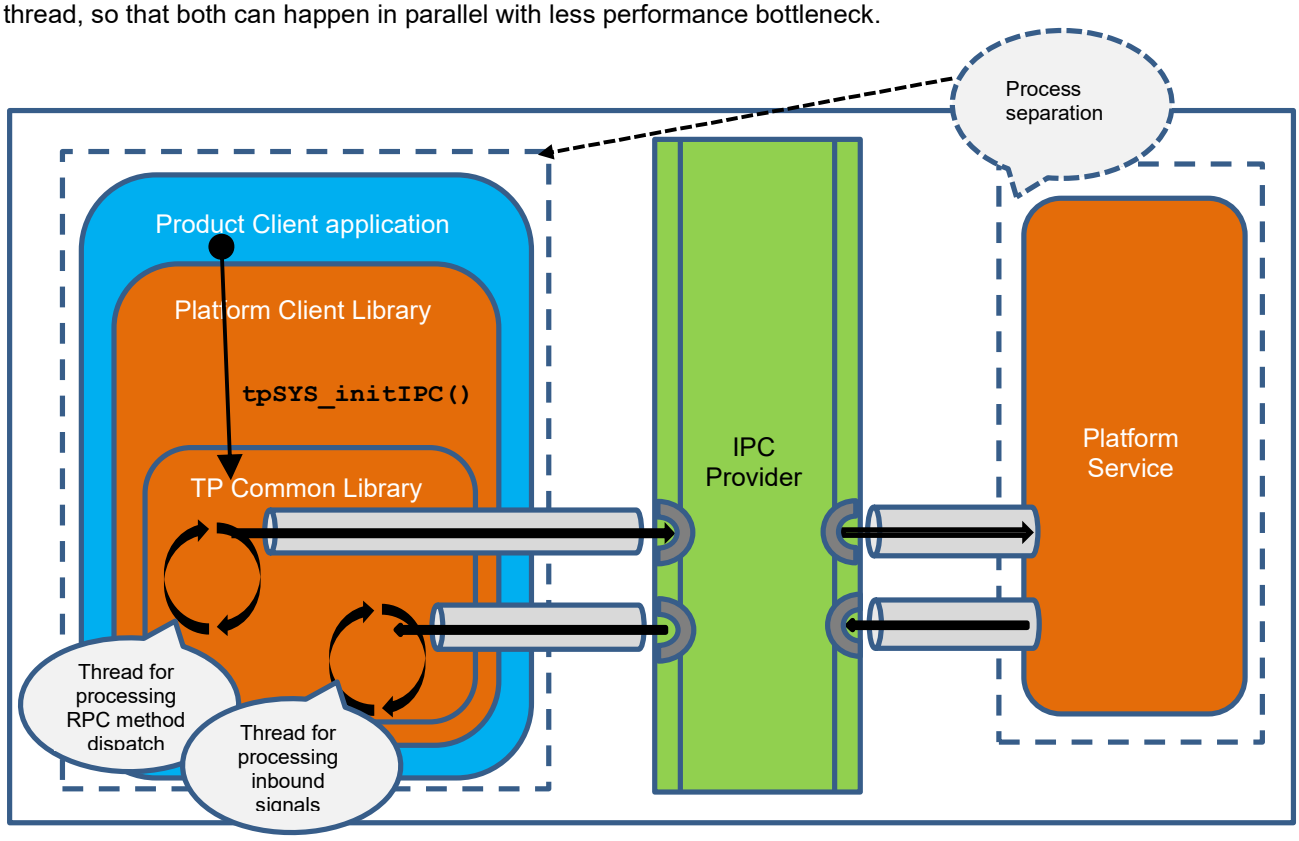
3 Features/Components

3.1 Platform IPC framework

3.1.1 Overview

3.1.1.1 Platform Content Overview

Platform provides APIs so that an application can run as IPC client easily. A threaded approach is taken for handling incoming notifications (signals) in one thread and invoking Remote Procedue Call (RPC) methods from another thread, so that both can happen in parallel with less performance bottleneck.



3.1.1.2 Product Expectations

The following expectations must be met by products (users of the platform) in order for this feature to work as designed:

1) Product must initialize the IPC setup using the tpSYS_initIPC() API before using API's from Platform Client Libraries, as documented..
2) Product must clean the IPC setup, using the tpSYS_deinitIPC() API, in order to properly stop IPC message handling threads and in order to clean up resources. The deinit API need to be called when interaction with the Platform Client Libraries are done.
3) Product must ensure that, tpSYS_initIPC() calls must be paired with tpSYS_deinitIPC() calls; even if tpSYS_initIPC() fails, tpSYS_deinitIPC() must be called, before retrying init again. Also, within a specific process context parallel calls to tpSYS_(de)initIPC() must be avoided.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

3.1.1.3 Extending Platform Functionality

Not Applicable.

3.1.2 Setup

Not Applicable.

3.1.2.1 Tools/Equipment Needed

Not Applicable.

3.1.2.2 Setup Instructions

Not Applicable.

3.1.2.3 Setup Troubleshooting

Not Applicable.

3.1.3 Usage

3.1.3.1 Basic Instructions

Not Applicable.

3.1.3.2 Detailed Instructions

References to TP common APIs and types that can be found in this section. For more details on these APIs and types see the Doxygen links in the following table:

API/Type Description	Doxygen Path

Enumeration of platform supported IPC techniques.	`Modules`
	`|-> Operating System (SYS)`
	`|-> tpSYS_Ipc_t`

API to setup IPC	`Modules`
	`|-> Operating System (SYS)`
	`|-> tpSYS_initIPC()`

API to cleanup IPC	`Modules`
	`|-> Operating System (SYS)`
	`|-> tpSYS_deinitIPC()`

API to reload configuration for IPC	`Modules`
	`|-> Operating System (SYS)`
	`|-> tpSYS_reloadConfigIPC()`

The following are the fundamental aspects of this feature:

1) Applications use *`tpSYS_initIPC()`* API to setup the application as a RPC client. When connection argument(s) are passed as NULL, platform sets up the respective connection to exchange RPC messages, and sets up individual threads for handling the RPC message over each connection.
   The argument ***`rx_conn`***, is used for handling inbound RPC messages (Typically in the thread "IPCThread_signal"). Since this connection is used to process inbound "signal"s, it's named as receiver connection.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

The argument ***tx_conn***, is used for handling outbound RPC messages, those are “asynchronous RPC method calls” (Typically in the thread “IPCThread_method”). Since this connection is used to process outbound RPC, its named as transmitter connection.

Product may choose to setup both the rx and tx connections through the platform provided IPC setup API, or any one (even none) rx or tx connection through the platform setup; Product may choose to have their own connection for handling signals and methods, using their own handling strategy. This can be achieved by passing non NULL (already existing) references to *the RPC connection parameters*(rx or tx)of the init IPC API. When a NULL reference is passed, then only platform sets up the respective connection and handling thread for it, otherwise the Product provided connection is used.

2) Applications use *tpSYS_deinitIPC()* API to cleanup the IPC setup (with cleaning up of any worker threads) which was done by *tpSYS_initIPC()*

3) Applications use *tpSYS_reloadConfigIPC()* API to force load IPC configuration files dynamically.

Note: This *tpSYS_reloadConfigIPC()* API is provided for special purpose to overcome certain class of IPC errors.

Example: Typically with the choice of IPC framework being DBus, for the issues like DBus service name lost, it requests the DBus daemon to reload all the configuration files for the services registered with DBus.

3.1.3.3 Testing/Error Reporting

Not Applicable.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

3.2 Platform service monitor framework

3.2.1 Overview

3.2.1.1 Platform Content Overview

Service monitor library provides :

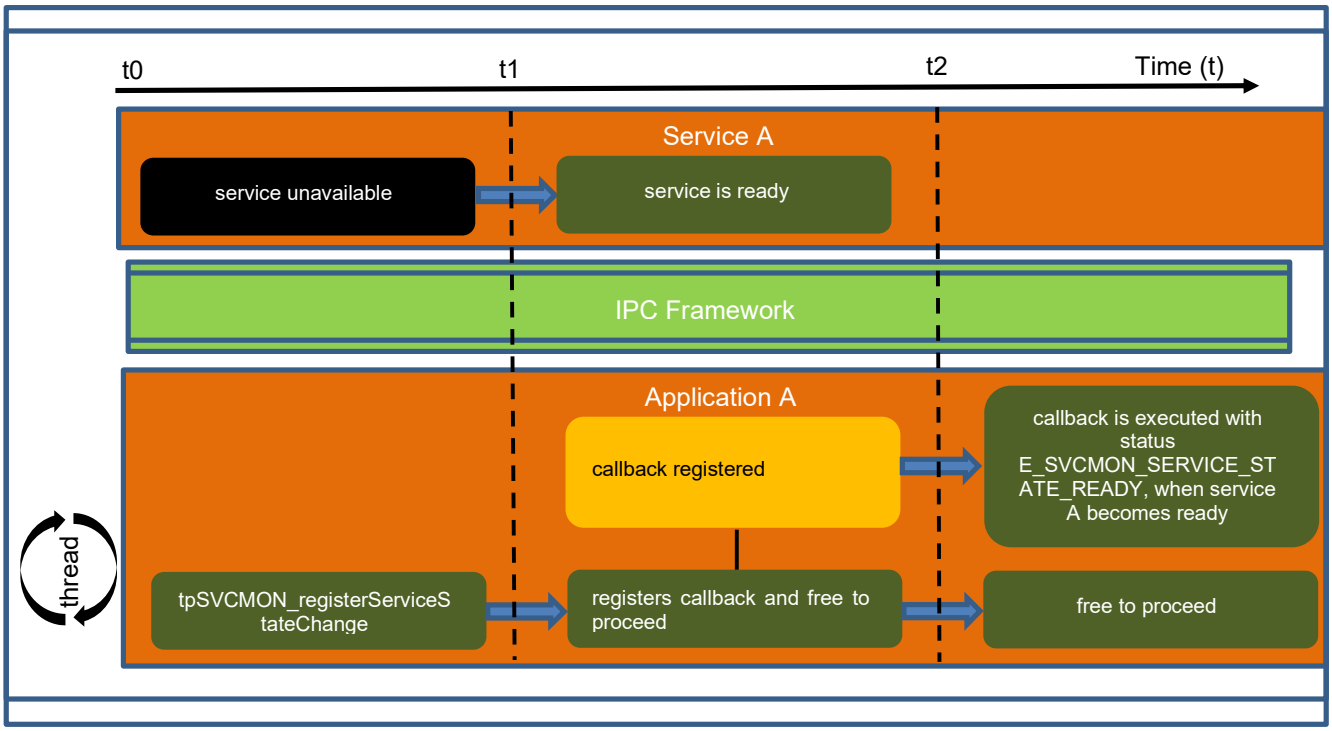
- APIs for service lifecycle event notifications (such as when the service is ready for communication) from both “local” and “remote” application processor;
- APIs to start/stop/restart services on “local” and “remote” application processor;
- APIs for boot state event notifications (such as booting / boot-complete / boot-complete-with-errors) from both “local” and “remote” application processor.

Services are managed by Systemd. With the platform provided service startup notification mechanism, it is possible to start any service irrespective of the dependency order imposed by the static systemd service configuration files.

If a service is dependent on another service, but part of it can run independently, then, with this facility the dependent services also can start in parallel, run the independent parts and can still be waiting for the dependent parts until the other service is ready.

This provides service startup performance improvement, particularly during boot – when all the services are started, by starting services in parallel when resources are available. Services that are multi-threaded can leverage this benefit significantly by starting independent threads in parallel, processing important early startup tasks therein, and blocking on the threads where there is indeed some dependency to another service. Also the dynamic notification facility provides the option to be notified when a service is ready, and act further using this as an event; this does not even block any thread.

This also provides flexibility in writing static systemd service configuration files, by removing service dependencies hardcoded, when those are not extremely needed.



© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

Service monitor abstracts details from applications so that they can monitor services on External Application Processor (extAP) or Modem Application Processor (modemAP) seamlessly. The boot state monitoring APIs provide applications the capability to be aware when the system (local or both processors) is completely booted.

Features:

1. Other product specific services or open source services can also be monitored/controlled using service monitor APIs.
2. The `tpSVCMON_registerServiceStateChange()` API support waiting for a number of services in a single call. The user can choose to be notified either for a particular service or for several services whose names match with the given service name prefix.
3. It is possible to precisely control actions when a listener is notified after a service starts up. Because of the presence of callback notification, actions can be dynamically and precisely controlled when a service is available – which is not possible using static dependency in systemd service config files.
4. It is possible to specify service dependencies at runtime driven by logic, by changing the argument which takes the name of service to wait for. This dynamic behaviour is not possible with static systemd service config files.
5. It is possible to allow packages to install configuration files of a specific format at a predefined location `(/opt/conti/etc/svcmon/*.conf)` in order to consolidate different service names (Dbus and systemd service names) into a singular Telematics Platform service name. Products could use this configuration to install well-known names for open-source services and/or product specific applications/services.
6.

Limitations:

1. Service Monitor compiled for Modem AP will only monitor "local" services and receive boot notification only for “local” bootup because TP Fermi Architecture client/server model is uni-directional (only External AP client → Modem AP server) and not reverse.
2. If both “local” and “remote” service have the same name, only the “local” service is monitored.

3.2.1.2 Product Expectations

The following expectations must be met by products (users of the platform) in order for this feature to work as designed:

1) Product must first initialize service monitor using `tpSVCMON_init()` API, in order to be able to use these APIs.
2) All applications should have a startup dependency (systemd "`After=service-monitor.service`") on TP Service Monitor.
3) Client applications should re-initialize communication with a service when transitioning from “Not Ready” state to “Ready” state.
4) In the event of service monitor service crash/restart, the client application should re-initialize communication with the service monitor and re-register for all asynchronous callbacks.

3.2.1.3 Extending Platform Functionality

Not Applicable.

3.2.2 Setup

Not Applicable.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

3.2.2.1 Tools/Equipment Needed

Not Applicable.

3.2.2.2 Setup Instructions

Not Applicable.

3.2.2.3 Setup Troubleshooting

Not Applicable.

3.2.3 Usage

3.2.3.1 Basic Instructions

Not Applicable.

3.2.3.2 Detailed Instructions

References to TP service monitor APIs and types that can be found in this section. For more details on these APIs and types see the Doxygen links in the path:

```
Modules
      |-> System (SYS)
            |-> Service Monitor (SVCMON)
```

API/Type Description	Doxygen Element
Service state change notification callback type	`tpSVCMON_ServiceStateCallback_t`
API to register for service state change notification	`tpSVCMON_registerServiceStateChange()`
API to get service state	`tpSVCMON_getServiceState()`
Boot state change notification callback type	`tpSVCMON_BootStateCallback_t`
API to register for boot state change notification	`tpSVCMON_registerBootStateChange()`
API to get boot state	`tpSVCMON_getBootState()`
API to start a service	`tpSVCMON_startService()`
API to stop a service	`tpSVCMON_stopService()`
API to restart a service	`tpSVCMON_restartService()`

The following are the fundamental aspects of this feature:

1) Applications can use `tpSVCMON_registerServiceStateChange()` API to register a callback to be asynchronously called when a service state (refer below) change. This API does not block the caller; instead the listener has the flexibility to be notified dynamically. When the is_prefix argument is set the notification can come for a number of services whose names match with the provided service_name as a prefix string. Applications can also use `tpSVCMON_getServiceState()` API to fetch the current state of a service.
2) Applications can use `tpSVCMON_registerBootStateChange()` API to register a callback to be asynchronously called when the boot state changes for “local” or “remote” application processor. Applications can also use `tpSVCMON_getBootState()` API to fetch the current boot state.
3) Applications can use `tpSVCMON_startService()` API to start a “local” or “remote” service. Similarly, `tpSVCMON_stopService()` and `tpSVCMON_restartService()` can be used to stop/restart a “local” or “remote” service.

© Continental Automotive GmbH 2019
Confidential and Proprietary

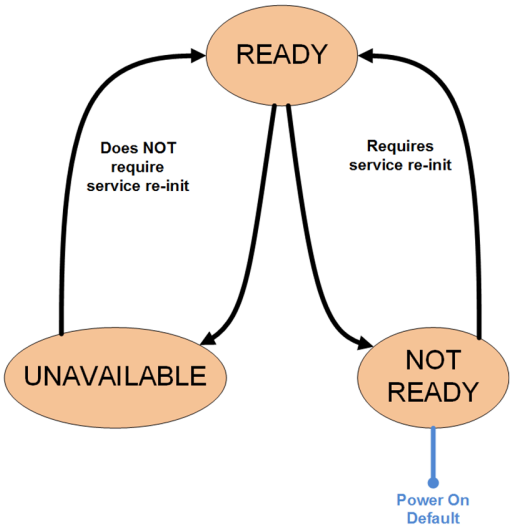
Sensitivity: Internal & Restricted

4) Applications have the benefit of using consolidated service names instead of full DBus or systemd names. A configuration file can be specified for each service and applications that can use the following service monitor APIs with the consolidated names:
    - `tpSVCMON_registerServiceStateChange()`
    - `tpSVCMON_getServiceState()`
    - `tpSVCMON_startService()`
    - `tpSVCMON_stopService()`
    - `tpSVCMON_restartService()`

    The configuration file can accept a single *"DBusName"* , a single *"SystemdName"* , or both under the well-known Telematics Platform name which will act as the consolidated name. TP Service Monitor configuration files conform to the GKeyFile (https://developer.gnome.org/glib/stable/glib-Key-value-file-parser.html) .ini style format. The first Group enclosed in brackets '[' and ']' is the consolidated service name that should be used in all TP Service Monitor public APIs.

5)

Service State Machine



READY: State indicating that a service is ready for communication and query/control.

If transitioning from "Not Ready" state, registered clients should re-initialize communication with the service and re-register for any asynchronous notifications.

NOT READY: State indicating that a service is not ready for communication and query/control.

Registered clients should tear down (un-initialize) communication channel(s) with the service upon receiving this indication.

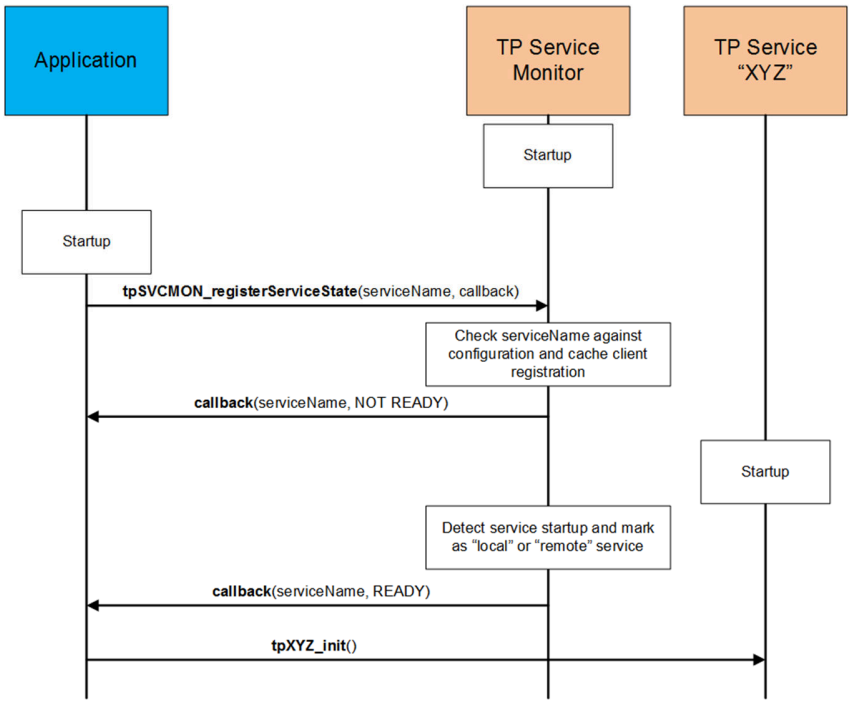
UNAVAILABLE: State indicating that a service is temporarily unavailable for communication and query/control.

Registered clients do not have to tear down communication with the service in this state

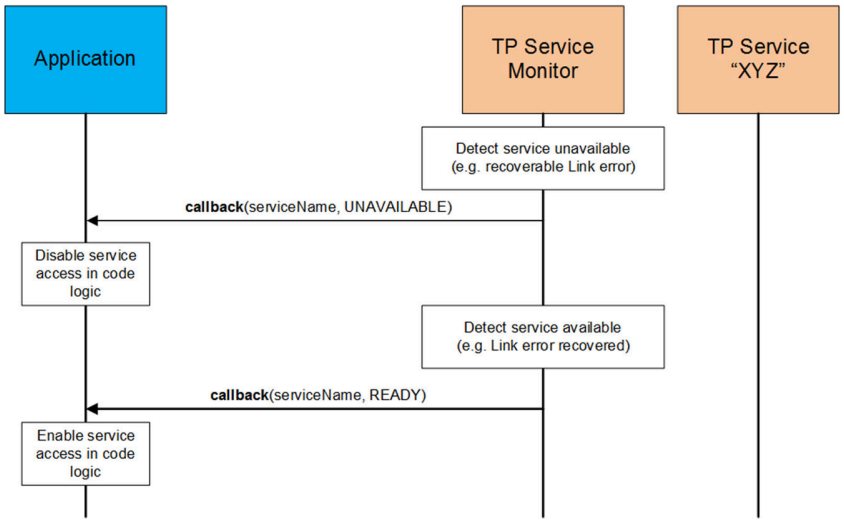
© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

Service Startup

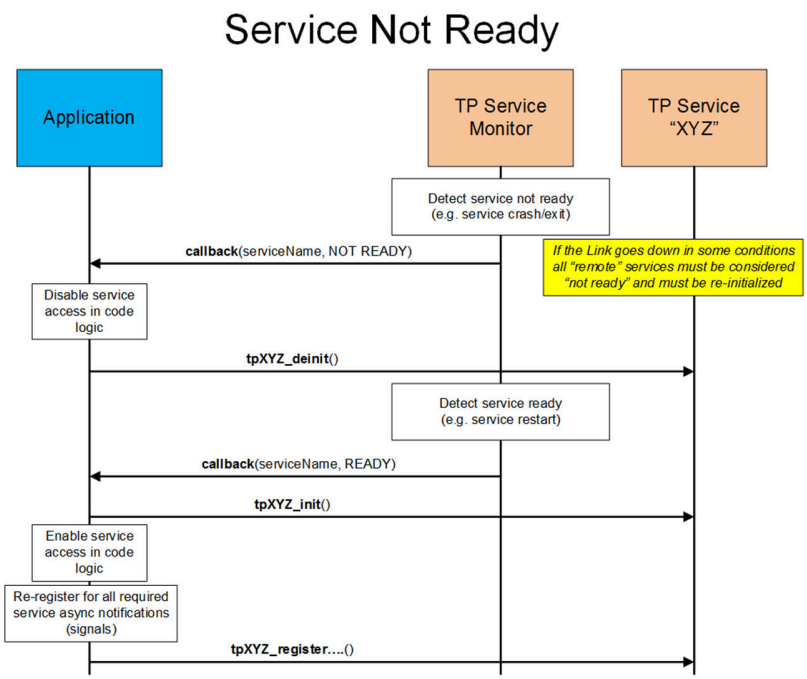


Service Unavailable



© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted



Code Example: Wait for a service to be ready

```
/* Includes */
#include <pthread.h>
#include "tpsys.h"
#include "tpSVCMON.h"

/* Prototypes */
static void handle_service_state_cb(const gchar* service_name, const tpSVCMON_ServiceState_t _state);
static void waitForServiceReady(void);

/*Local Variables*/
static gboolean service_ready = FALSE;
static GMutex service_ready_mutex;
static GCond service_ready_cond;

/* main */
int main (int argc, char* const argv[])
{
      gint monitorID = 0;
      /* Init service monitor:
         Note: It only needs to be called if it was not initialized in the
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

```
          client process yet */
      if(tpSVCMON_init() != E_SVCMON_ERROR_NONE){
            /* Unable to initialize SVCMON */
            return 1;
      }

      /* register a callback for the dependent service */
      tpSVCMON_registerServiceStateChange(DEP_SERVICE_NAME, FALSE,
                  handle_service_state_cb, &monitorID);

      /* Placeholder for application logic independent of the DEP_SERVICE */

      waitForServiceReady();

      /* Placeholder for application logic dependent on the DEP_SERVICE */

      tpSVCMON_deinit();

      return 0;
}

static void waitForServiceReady() {
      g_mutex_lock(&service_ready_mutex);
      while (!service_ready) {
            g_cond_wait(&service_ready_cond, &service_ready_mutex);
      }
      g_mutex_unlock(&service_ready_mutex);
}

static gpointer client_thread(gpointer data)
{
      g_mutex_lock (&service_ready_mutex);
      service_ready = is_ready;
      g_cond_signal (&service_ready_cond);
      g_mutex_unlock (&service_ready_mutex);
      return NULL;
}

static    void    handle_service_state_cb(const    gchar*    service_name,    const
tpSVCMON_ServiceState_t _state){
      static          tpSVCMON_ServiceState_t          previous_state          =
E_SVCMON_SERVICE_STATE_NOT_READY;
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

```
        /* Service Monitor calls this callback from tp-common socket client thread
         * It's not recommended to do much processing in this function
         * Better to create another thread for the work */
        if(E_SVCMON_SERVICE_STATE_READY == _state){
              if(E_SVCMON_SERVICE_STATE_NOT_READY == previous_state){
                    g_thread_unref(g_thread_new("client_thread",
(GThreadFunc)client_thread, NULL));
              }
        }
        previous_state = _state;
}
```

3.2.3.3 Testing/Error Reporting

Not Applicable.

3.3 Miscellaneous Interfaces for debugging

3.3.1 Overview

3.3.1.1 Platform Content Overview

Common library provides APIs for printing stack traces for debugging purpose. Also a custom signal handler is provided to be installed for certain signals, so that, stack trace is printed when the application receives the signal.

Also the common library provides API to setup watchdog for a systemd service.

Please refer: http://man7.org/linux/man-pages/man3/backtrace.3.html for understanding backtrace (with symbols). In general, to gather more information on debugging stack traces please refer links provided in section 7.3,

Refer http://man7.org/linux/man-pages/man7/signal.7.html for signal handlers. For details on Linux signal model please refer the links provided in section 7.3, for Linux signal handling concepts.

Refer https://www.freedesktop.org/software/systemd/man/sd_notify.html for watchdog usage with systemd.

3.3.1.2 Product Expectations

Not Applicable.

3.3.1.3 Extending Platform Functionality

Products can use their own implementation for the functionalities mentioned in the above section; however it is strongly recommended that platform provided API's are instead used in order to maintain uniform behavior and for ease in debugging.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

3.3.2 Setup

Not Applicable.

3.3.2.1 Tools/Equipment Needed

Not Applicable.

3.3.2.2 Setup Instructions

Not Applicable.

3.3.2.3 Setup Troubleshooting

Not Applicable.

3.3.3 Usage

3.3.3.1 Basic Instructions

Not Applicable.

3.3.3.2 Detailed Instructions

References to TP common APIs and types that can be found in this section. For more details on these APIs and types see the Doxygen links in the following table:

API/Type Description	Doxygen Path

API to print call stack	`Modules`
	`|-> Operating System (SYS)`
	`|-> tpSYS_stackTrace()`

API to print call stack for a particular Linux signal received	`Modules`
	`|-> Operating System (SYS)`
	`|-> tpSYS_stackTraceSignalHandler()`

API to setup watchdog for service alive heartbeat	`Modules`
	`|-> Operating System (SYS)`
	`|-> tpSYS_setupWatchdog()`

The following are the fundamental aspects of this feature:

1) Applications can use `tpSYS_stackTrace()` in exception handlers to print call stack with symbols. The primary usage is for debugging.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

2) Applications can use `tpSYS_stackTraceSignalHandler()` API to use as a custom signal handler to print call stack with symbols. The primary usage is for debugging. The implementation will invoke the default signal handler for the signal after this handler is finished. This API should be used for SIGSEGV and SIGABRT typically, as custom signal handler. It prints stack trace with ERROR log level; so, it may not be desirable to log as errors in other signal handlers.

3) Applications can use `tpSYS_setupWatchdog()` API to configure the service's keep alive notification to the systemd service manager at specified periodic interval, when such service heartbeat is enabled in service's systemd configuration file. (If the service manager fails to receive keep-alive notifications for a particular service, configured recovery actions will be initiated.)

Note: The caller process of this API need to have a running g_main_loop with the global default main context. If tpSYS_initIPC() is used, this requirement is taken care of already.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

4 Known Issues

Not Applicable.

5 FAQs

Not Applicable.

6 Troubleshooting/Tips and Tricks

Not Applicable.

7 References

7.1 Abbreviations/Acronyms/Definitions

RPC	*Remote Procedure Call*

7.2 Platform Documentation

TP_102G-00001: Telematics Platform Glossary

7.3 Website links

https://developer.gnome.org/glib/
https://developer.gnome.org/glib/stable/glib-The-Main-Event-Loop.html
https://developer.gnome.org/gio/stable/gdbus-lowlevel.html
https://www.freedesktop.org/wiki/Software/systemd/
https://sourceware.org/gdb/download/onlinedocs/gdb/Stack.html#Stack
http://infocenter.arm.com/help/index.jsp?topic=/com.arm.doc.subset.swdev.abi/index.html
https://www.gnu.org/software/libc/manual/html_node/Signal-Handling.html
https://developer.gnome.org/glib/stable/glib-Key-value-file-parser.html

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

8 Revision History

Version	Date	Description
01.00	Aug 14, 2018	Initial release
02.00	Jul 10, 2020	Add service monitor details
03.00	Oct 01, 2020	Minor changes to service monitor features
		== ConMod-specific content from here ==

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted